Architecture des Ordinateurs : Représentation des Entiers et Arithmétique en Complément à Deux

Thème principal : Représentation binaire des entiers signés, algorithme du complément à deux, arithmétique de l'ALU et détection de l'Overflow

Synthèse exécutive : Ce cours aborde la manière dont les ordinateurs codent et manipulent physiquement les entiers signés et non signés en mémoire binaire sur une largeur fixe de N bits. Face à l'impossibilité matérielle des composants électroniques (transistors, bascules bistables) de stocker un symbole abstrait '+' ou '-', l'architecture informatique moderne repose universellement sur la convention du complément à deux. Cette méthode élimine l'inconvénient du double zéro présent dans la représentation 'Signe et Magnitude' et unifie les opérations d'addition et de soustraction au sein du même additionneur matériel dans l'Unité Arithmétique et Logique (ALU). La séance détaille l'algorithme de conversion en 3 étapes, la dissymétrie des plages de valeurs représentables [-2^(N-1), +2^(N-1)-1], la distinction fondamentale entre la retenue sortante (Carry Out) et le dépassement de capacité (Overflow), ainsi que les critères formels de détection des erreurs arithmétiques bas niveau.

1. Problématique matérielle et justification de la convention du Complément à Deux

Au niveau le plus bas de la hiérarchie matérielle d'un ordinateur, la mémoire vive et les registres processeur sont constitués de composants électroniques bistables ne pouvant enregistrer que deux niveaux de tension logique : 0 Volt (niveau logique 0) et la tension d'alimentation Vcc (niveau logique 1). Un microprocesseur ne possède aucun moyen physique de stocker un caractère abstrait comme '+' ou '-'. Pour coder des entiers relatifs (positifs et négatifs), les architectes ont d'abord expérimenté le codage 'Signe et Magnitude', où le bit le plus à gauche (MSB) indique le signe (0 pour positif, 1 pour négatif) et les bits suivants portent la valeur absolue. Cette approche présente deux faiblesses majeures : elle crée deux représentations distinctes pour la valeur zéro (+0 = 0000_2 et -0 = 1000_2 sur 4 bits), ce qui gaspille une combinaison binaire utile et alourdit la logique matérielle lors des tests d'égalité ; d'autre part, elle oblige à construire deux circuits séparés dans l'Unité Arithmétique et Logique (ALU), l'un pour additionner et l'autre pour soustraire. La convention du complément à deux résout intégralement ces contraintes. Le nombre zéro y possède une signature binaire unique (tous les bits à 0). De plus, l'opération de soustraction A - B s'effectue sous la forme d'une addition A + (-B), réutilisant directement l'additionneur binaire standard de l'ALU sans composant additionnel.

- • Absence de signe physique dans les circuits logiques : les transistors et bascules bistables ne stockent que deux états électriques (0V pour 0, Vcc pour 1).
- • Limites du codage Signe et Magnitude : présence d'un double zéro (+0 et -0) entraînant le gaspillage d'une combinaison binaire et nécessitant deux circuits matériels séparés dans l'ALU.
- • Unification ALU par le complément à deux : unicité de la représentation du zéro (0000...0) et calcul des soustractions A - B sous la forme d'additions A + (-B) sur un additionneur unique.

Cas concret enseignant : *La métaphore du compteur kilométrique d'une voiture ancienne à 4 chiffres : si le compteur affiche 0000 km et que vous effectuez un kilomètre en marche arrière, le compteur repasse mécaniquement à 9999. En binaire sur 8 bits, faire 00000000_2 - 1 donne 11111111_2, ce qui constitue la définition formelle de -1 en complément à deux.*

2. Algorithme de Conversion et Plage de Valeurs sur N bits

Pour convertir un entier négatif -X en complément à deux, il est impératif de définir au préalable la largeur N du mot mémoire (ex: 8, 16, 32 ou 64 bits). La méthode exige d'exécuter rigoureusement trois étapes dans l'ordre : 1) Convertir la valeur absolue X en binaire naturel et la cadrer sur N bits en complétant par des 0 sur la gauche ; 2) Inverser la totalité des bits (les 0 deviennent 1 et les 1 deviennent 0), ce qui produit le complément à un ; 3) Ajouter 1 au mot binaire obtenu pour former le complément à deux. Le bit le plus à gauche (MSB) vaut 0 pour un nombre

positif ou nul, et 1 pour un nombre négatif. En raison du rattachement du zéro (0000...0, dont le MSB vaut 0) au domaine des nombres positifs, la plage de valeurs représentables sur N bits est dissymétrique : la plage comporte toujours une valeur négative de plus que de valeurs positives, s'étendant formellement de -2^(N-1) à +(2^(N-1) - 1). Sur 8 bits, la plage s'étend ainsi de -128 à +127.

- Algorithme séquentiel en 3 étapes : Valeur absolue sur N bits -> Complément à 1 (inversion de tous les bits) -> Complément à 2 (ajout de 1).
- Rôle du Most Significant Bit (MSB) : le bit le plus à gauche agit comme indicateur de signe (0 = positif ou nul, 1 = négatif).
- Plage de valeurs dissymétrique : de -2^(N-1) jusqu'à +(2^(N-1) - 1) sur un mot de N bits (ex: [-128, +127] sur 8 bits).

Cas concret enseignant : *Piège classique d'évaluation souligné par l'enseignant : il faut IMPÉRATIVEMENT fixer la taille N bits (ex: 8 bits) et écrire les zéros de cadrage à gauche AVANT d'effectuer l'inversion des bits. Inverser la séquence '1100' (+12 sur 4 bits) sans ajouter au préalable les 4 zéros de cadrage '00001100' donne '0011' au lieu de '11110011', ce qui conduit à un résultat totalement faux en contrôle !*

3. Arithmétique, Retenue (Carry Out) et Détection du Dépassement de Capacité (Overflow)

Lors de l'exécution d'une addition binaire dans un registre processeur de taille fixe N, deux indicateurs logiques distincts doivent être analysés au niveau du registre d'état (drapeaux/flags) : le Carry Out (C) et l'Overflow (OV). Le Carry Out représente la retenue sortante générée au-delà du bit MSB (à la position N+1). En arithmétique signée en complément à deux, cette retenue sortante est rejetée car le processeur effectue ses opérations en arithmétique modulaire de période 2^N. L'Overflow, en revanche, signale une erreur de dépassement de capacité où le bit de signe a été corrompu par le calcul arithmétique. La règle d'évaluation formelle pour détecter la présence d'un Overflow est stricte : un Overflow se produit si et seulement si deux opérandes ayant le même bit de signe (deux positifs ou deux négatifs) sont additionnés et donnent un résultat avec un bit de signe opposé. Si les deux opérandes sont de signes différents (un positif et un négatif), la survenue d'un Overflow est mathématiquement et physiquement impossible.

- Elimination du Carry Out : la retenue sortant du bit MSB est systématiquement ignorée par l'ALU car le processeur calcule modulo 2^N.
- Définition de l'Overflow (OV) : erreur d'amplitude indiquant que le résultat mathématique excède la capacité du registre [-2^(N-1), +2^(N-1)-1].
- Règle d'or de détection de l'Overflow : un Overflow survient si et seulement si l'addition de deux opérandes de même signe produit un résultat de signe opposé.

Cas concret enseignant : *L'anecdote historique du crash d'Ariane 5 (vol 501 en 1996) : une conversion de type d'un nombre flottant sur 64 bits vers un entier signé sur 16 bits sans protection contre le dépassement de capacité a provoqué un Overflow non capturé. L'Unité de Référence Inertielle a transmis une valeur corrompue au calculateur de bord, interprétée à tort comme une déviation de trajectoire, provoquant le braquage brutal des tuyères puis l'autodestruction de la fusée.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la retenue tout à gauche (Carry sortant du bit MSB) est-elle systématiquement jetée lors de l'addition en complément à deux ?
Réponse du professeur : Parce que le microprocesseur effectue ses opérations arithmétiques sur des registres de taille fixe N bits, travaillant dans un anneau d'arithmétique modulaire modulo 2^N. La retenue à la position N+1 sort physiquement du registre. Mathématiquement, ignorer cette retenue revient à appliquer l'opération modulo 2^N, ce qui conserve exactement le résultat correct dans la représentation du complément à deux sans altérer sa valeur.

Question étudiant : Comment l'ordinateur fait-il la différence en mémoire entre le mot binaire 11111111 qui vaut 255 en non-signé et -1 en complément à deux ?
Réponse du professeur : Physiquement, l'ordinateur ne fait aucune différence : les 8 bascules mémoire contiennent exactement la même suite de 8 bits (11111111). C'est le programmeur, via la déclaration de type de la variable (par exemple 'unsigned char' pour 255 ou 'signed char' / 'int8_t' pour -1 en C/C++), qui donne du sens à ce motif binaire. Au niveau de l'assembleur, cela se traduit par le choix d'instructions spécifiques de comparaison ou d'arithmétique (instructions signées ou non signées).

■ Exercice & Démonstration d'Application

Énoncé : Sur une architecture 8 bits, effectuer la soustraction signée : 35 - 75. Détailler le codage binaire des deux opérandes en complément à deux, poser l'addition binaire correspondante, donner le résultat décimal final et analyser la valeur du drapeau d'Overflow (OV).

Démarche de résolution :
Étape 1 : Codage du premier opérande (+35) sur 8 bits. 35 en binaire pur = 32 + 2 + 1 = 00100011_2. MSB = 0 (nombre positif). Étape 2 : Codage du second opérande (-75) en complément à deux sur 8 bits. - Valeur absolue (+75) sur 8 bits : 64 + 8 + 2 + 1 = 01001011_2. - Inversion des bits (Complément à 1) : 10110100_2. - Ajout de 1 (Complément à 2) : 10110100 + 1 = 10110101_2 (Représentation de -75 sur 8 bits). Étape 3 : Pose et exécution de l'addition binaire (+35) + (-75). 00100011 (+35) + 10110101 (-75) ----------- 11011000_2 Étape 4 : Interprétation du résultat binaire 11011000_2. Le bit de poids fort MSB vaut 1, le résultat est négatif. Pour retrouver sa valeur absolue décimale, on réapplique le complément à deux sur 11011000_2 : - Inversion des bits : 00100111_2 - Ajout de 1 : 00100111 + 1 = 00101000_2 = 32 + 8 = 40 décimal. Le mot binaire 11011000_2 représente donc la valeur -40 en décimal. Étape 5 : Analyse du drapeau d'Overflow (OV). On additionne un nombre positif (+35, MSB=0) et un nombre négatif (-75, MSB=1). Les deux opérandes possédant des bits de signe différents, l'apparition d'un Overflow est mathématiquement impossible.

```
/*
  Nom du fichier : complement_deux.cpp
  Auteur         : Matéi de Saint Leger
  Date           : 12/11/2026
  Objet          : Soustraction signée sur 8 bits et vérification d'Overflow
*/

#include <iostream>
#include <cstdint>
using namespace std;

int main()
{
    // CONSTANTES

    // VARIABLES
    int8_t operandeA;
    int8_t operandeB;
    int8_t resultat;

    // TRAITEMENTS
    operandeA = 35;
    operandeB = -75;
    resultat = operandeA + operandeB;

    cout << "Opérande A (+35 décimal) : " << static_cast<int>(operandeA) << endl;
    cout << "Opérande B (-75 décimal) : " << static_cast<int>(operandeB) << endl;
    cout << "Résultat (35 - 75)      : " << static_cast<int>(resultat) << endl;

    return 0;
}
```

Résultat : Résultat binaire final : 11011000_2 (égal à -40 en décimal). Drapeau d'Overflow OV = 0 (aucun dépassement de capacité). Le calcul 35 - 75 = -40 est exact et validé.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la plage exacte des entiers signés représentables sur un mot mémoire de 16 bits en complément à deux ?

R : La plage s'étend de -2^(16-1) à +(2^(16-1) - 1), soit de -32 768 à +32 767.

Q : Comment détecte-t-on de façon certaine la présence d'un Overflow lors de l'addition de deux entiers signés en examen ?

R : Un Overflow survient si et seulement si l'on additionne deux opérandes de même signe (deux MSB à 0 ou deux MSB à 1) et que le résultat présente un bit de signe opposé. L'addition de deux nombres de signes différents ne peut jamais générer d'Overflow.

Q : Quel est le résultat de l'application de la procédure du complément à deux sur la valeur zéro (00000000_2) sur 8 bits ?

R : L'inversion des bits donne 11111111_2. L'ajout de 1 donne (1)00000000_2. La retenue sortante est rejetée (modulo 256), redonnant 00000000_2. Le zéro est donc invariant et possède une representation unique.